QUESTION :

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

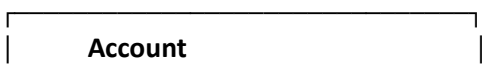
Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

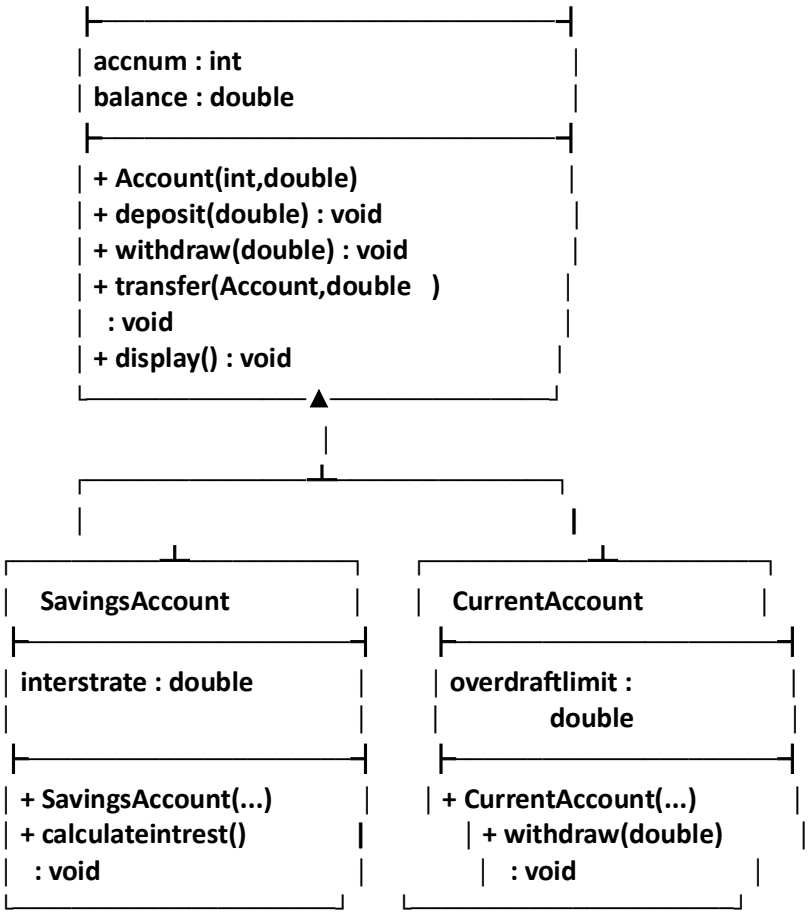
EXECUTION STEPS :

ALGORITHM :

1. **Start** the program.
2. Create a class `Account` with the attributes:
   - `accnum`
   - `balance`
3. Create a constructor in `Account` to initialize the account number and balance.
4. Create the following methods in `Account`:
   - `deposit()` to add money.
   - `withdraw()` to withdraw money if sufficient balance is available.
   - `transfer()` to transfer money from one account to another.
   - `display()` to display account details.
5. Create a `SavingsAccount` class that **inherits** from `Account`.
6. Add `interstrate` to `SavingsAccount` and create a constructor using `super()` to initialize the parent class.
7. Create `calculateintrest()` to calculate interest using:
   Interest = Balance × (Interest Rate / 100)
8. Create a `CurrentAccount` class that **inherits** from `Account`.
9. Add an `overdraftlimit` to `CurrentAccount` and **override** the `withdraw()` method to allow withdrawal up to the overdraft limit.
10. In the `main()` method, create:
    - A `SavingsAccount` object `s`
    - A `CurrentAccount` object `c`
11. Calculate interest for the savings account.
12. Deposit `5000` into the savings account.
13. Withdraw `2000` from the current account.
14. Calculate the interest on the savings account again.
15. Transfer `500` from the savings account to the current account.
16. Display the updated transaction results.
17. **Stop** the program.

UML DIAGRAM :





CODE :

```
class Account {
   int accnum;
   double balance;

   public Account(int accnum, double balance) {
      this.accnum = accnum;
      this.balance = balance;

   }

   void deposit(double amount) {

      balance = balance + amount;
      System.out.println("the total balance is :" + balance);
      System.out.println("Deposited: " + amount);

   }

   void withdraw(double amount) {
      if (balance >= amount) {
         balance = balance - amount;
         System.out.println("the total balance is :" + balance);
         System.out.println("Withdrawn: " + amount);
      } else {
```

```
            System.out.println("Insufficient balance");
        }
    }

    void transfer(Account receiver, double amount) {
        if (balance >= amount) {
            this.balance = this.balance - amount;
            receiver.balance = receiver.balance + amount;
            System.out.println("Transferred: " + amount);
            System.out.println("the balance after transferred is :" + this.balance);
        } else {
            System.out.println("Transfer failed . please try again.");
        }
    }

    void display() {
        System.out.println("accnum :" + accnum);
        System.out.println("balance :" + balance);
    }
}
class SavingsAccount extends Account{
    double interstrate;
    public SavingsAccount(int accnum,double balance,double intrestrate){
        super(accnum,balance);
        this.interstrate=intrestrate;
    }
    void calculateintrest() {
        double intrest = balance * (interstrate / 100);
        balance = balance + intrest;
        System.out.println("the total balance is :"+balance);
        System.out.println("value :"+intrest);
    }

}
class CurrentAccount extends Account{
    double overdraftlimit;
    public CurrentAccount(int accnum,double balance){
        super(accnum,balance);
        this.overdraftlimit=overdraftlimit;
    }
    @Override
    void withdraw(double amount){
        if ( amount <= balance+overdraftlimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("exceed overdraft limt");
        }
    }

}
public class Bank{
    public static void main(String []args) {
        SavingsAccount s = new SavingsAccount(3000,2424,50);
        CurrentAccount c = new CurrentAccount(2254,5845);
        System.out.println("savings account");
s.calculateintrest();
```

```
        System.out.println("\nDeposit");
        s.deposit(5000);
        System.out.println("\nwithdraw");
        c.withdraw(2000);
        System.out.println("\ninterstrate");
        s.calculateintrest();
        System.out.println("\ntransfer");
        s.transfer(c,500);


    }
}
```

OUTPUT :

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2020.1.4\lib\idea_rt.jar=57904" -
savings account
the total balance is :3636.0
value :1212.0

Deposit
the total balance is :8636.0
Deposited: 5000.0

withdraw
Withdrawn: 2000.0

interstrate
the total balance is :12954.0
value :4318.0

transfer
Transferred: 500.0
the balance after transferred is :12454.0

Process finished with exit code 0
```